Speculation-Aware Scheduling for Multi-Step Agent Workloads

Project Proposal | Systems for LLMs and AI Agents

Sierra Myers | Yuehua Xie | Raghusrinivasan Venkatesan | Ashish Kumar Panda

Objective and research question

We will study how speculative decoding interacts with concurrency, growing contexts, tool delays, and reusable key–value (KV) cache state under a fixed single-GPU memory budget. Our question is: *when should resources admit another ready agent step instead of supporting more speculative work for running requests?* The primary outcome is successful tasks completed per second within a predefined task-latency target (task goodput). Token throughput alone may hide lower task throughput caused by speculative memory, verification contention, or repeated prefill.

Workload, platforms, and scope

We will start with one multi-step, tool-using workload, preferably coding-agent traces with long shared system prompts/tool definitions and growing histories. Candidates are `trace-commons/agent-traces` and filtered Hermes agent traces; we will inspect schemas, licensing, tool coverage, and replayability before selection. Traces will supply prompts, tool calls, and dependencies. LangGraph agent/tool nodes or the OpenAI Agents SDK Runner will drive execution. Controlled replay will preserve step dependencies and inject tool delays; executable tasks will assess correctness. Long-prefix chatbots are a secondary workload.

Available hardware is an RTX 5070 Laptop GPU (12 GB) and a Mac with 24 GB unified memory, a 15-core CPU (reported: five super cores at 4.6 GHz, ten performance cores at 4.4 GHz), and a 16-core integrated GPU at 1.6 GHz with neural acceleration. We will verify device specifications and memory headroom. Candidate engines are vLLM and SGLang on supported backends, with llama.cpp/MLX as a baseline. Phase 1 will pin one runtime, compatible target–drafter pair, precisions, and revisions after a fit/control-access check; model sizes remain open. NVIDIA and Mac results will be reported separately.

Phase 1: baseline and bottleneck measurements

Implement FIFO admission of ready *model requests* into a continuously batching runtime; distinguish live-task concurrency, admitted-request concurrency, and runtime sequence/token limits. During tool calls, release request admission slots while preserving agent state. Explicitly configure whether idle state is retained, reclaimable, offloaded, or recomputed; caching and FIFO are independent choices. Retained state is not necessarily pinned.

Compare ordinary decoding, tuned fixed-length autoregressive (AR) speculation, and stock adaptive speculation where supported. AR drafting is the starting point. Account for target/drafter weights, both models' KV state, lookahead/tree reservations, and verification work; speculation need not improve throughput at every load.

Initial sweeps: contexts {512, 4,096, 16,384} tokens; output caps {32, 256, 1,024}; requests/task {1, 4, 8}; tool gaps {0, 0.1, 1, 5} s. Adapt infeasible settings to capacity. Vary concurrency, unique/shared prefixes, conversation growth, steady/bursty arrivals, and abrupt load changes individually before combinations; add heavy-tailed delays later. Record actual output lengths. Select configurations on development traces and reserve held-out mixes.

Phase 2: focused scheduling intervention

If KV capacity limits task goodput, implement *speculation-aware admission*: estimate incremental physical KV demand after shared-prefix reuse, near-term context growth, and speculative reservations; admit with a calibrated safety margin, refresh estimates, and retain runtime allocation checks as final authority. Bound bypass and apply aging to prevent starvation. Keep eviction and the stock speculative controller fixed initially to isolate the policy's contribution.

Required controls are FIFO at reference concurrency; best fixed concurrency without speculation; best fixed concurrency with fixed speculation; stock adaptive speculation; and simple KV-aware admission using the same adaptive controller. Request-length prediction, tool-return prediction (InferCept-like), and draft-length adaptation are distinct decisions.

Extensions, only if measurements justify them and time allows, include joint admission/speculative budgets; progress-aware priorities with fairness; and tool-gap-aware retention versus LRU/TTL. Prefer useful-context retention; compare CPU/SSD offload and MiKV-style mixed-precision KV compression, including conversion/transfer overhead and task-quality loss. Dynamic KV quantization requires runtime/kernel support; changing weight precision is separate. Semantic eviction, overload rejection (Mooncake-like), and tool-call classifiers remain optional, not core commitments.

Evaluation, memory accounting, and deliverables

Use open-loop arrivals, dependency-preserving steps, repeated runs with confidence intervals, and pinned models/tokenizers/templates/configurations. Measure task completion time (p50/p95/p99), application/runtime queueing, prefill/decode and first-token latency, throughput/goodput, and deadline attainment. Log draft/verification time, proposed/accepted/committed tokens, batch size, prefix survival, evictions/preemptions, and recomputed tokens. Forced-output replay cannot establish acceptance or task-quality gains.

Separate infrastructure errors, admission rejections, cancellations, and timeouts from task failures (incorrect answers or failed task tests). Define peak memory as maximum sampled device/process allocation during a measured run, with sampling interval and allocation/reservation semantics disclosed. Separately report weights, workspace/graphs, active KV, reclaimable blocks, speculative reservations, and free KV headroom: a preallocated pool can mask pressure behind flat process memory. Smaller draft length need not release weights or all reserved memory. On Mac, distinguish unified process/GPU allocations from system memory; use exposed runtime/OS counters and Metal Instruments where available, and Nsight Systems on NVIDIA. Profilers supplement runtime cache counters; unavailable controls/measurements limit claims.

Ablate concurrency, speculation, cache capacity, reuse, and delays. Continue only if speculation-specific accounting improves the latency–throughput frontier; pivot to verification scheduling or retention if those dominate, and report negative results when simple controls match.

Selected related work

MiKV: Mixed-Precision KV Cache (compression); InferCept (tool-gap retention); vLLM speculative decoding and SGLang speculative decoding (stock baselines).